Improved pseudorandom generators for constant-depth circuits

Xin Lyu

doi:10.4230/LIPIcs.CCC.2022.34; CC BY4.0

Editorial note. Original author: Xin Lyu. Official CCC2022 article34, DOI10.4230/LIPIcs.CCC.2022.34.

Editorial note. License: https://creativecommons.org/licenses/by/4.0/

Editorial note. Faithful editable TeX transcription of the official CC BY4.0 main Theorem1, complete Section4 partitioning/hybrid construction and stronger Theorem10 proof, complete Sections5/6 switching and multi-switching proofs including Algorithms1–4 and all witness/searcher/decoupling continuations. Required author definitions, exact cited standard CNF PRG/hash facts and full bibliography retained. Only the formal published version is used; arxiv2301.10102 and Berkeley report are identity aliases, not content sources. Literal published expressions and numbering preserved; no mathematical corrections, added assumptions or new proofs. All supporting switching/partition results and bottom-width parameter refinement are grouped with the main PRG result.

Allowed standard prerequisites (editorial).

- CNF/DNF pseudorandom generator. Exact author Theorem8[5,26] retained.
- Bounded-independent hash families. Exact author Lemma6[28,Chapter3.5.5] retained.

Pseudorandom generator (author Introduction). Let $\mathcal{F}$ be a class of functions. A pseudorandom generator (PRG) for $\mathcal{F}$ is an algorithm $G : \{0,1\}^s \to \{0,1\}^n$ that maps a short random seed x into a longer string $G(x)$ that appears random to every distinguisher in $\mathcal{F}$. More specifically, we say that G ϵ-fools $\mathcal{F}$, if for every $f \in \mathcal{F}$, it holds $|\mathbb{E}_{x\sim U_s}[f(G(x))] - \mathbb{E}_{x\sim U_n}[f(x)]| \leq \epsilon$.

Author preliminaries (Section 3)

We use the term "restriction" and "partial assignment" interchangeably. Both of them refer to a string of the form $\rho \in \{0,1,\star\}$. Here, if $\rho_i = 0/1$, it means the i-th bit of ρ is fixed to that value. Otherwise, the i-th bit of ρ is unfixed. For two partial assignments $\rho, \sigma \in \{0,1,\star\}$, define their composition $\rho \circ \sigma$ as:

$$(\rho \circ \sigma)_i = \begin{cases} \rho_i & \rho_i \neq \star \\ \sigma_i & \text{o.w.} \end{cases}.$$

Note that the left partial assignment always has a higher priority than the right one. Let $\Lambda \subseteq [n]$ be a set. For two partial assignments $\rho, \sigma \in \{0,1,\star\}^n$, let $\Lambda[\rho,\sigma]$ be the assignment defined as

$$\Lambda[\rho,\sigma]_i = \begin{cases} \rho_i & i \notin \Lambda \\ \sigma_i & i \in \Lambda \end{cases}.$$

Let $f : \{0,1\}^n \to \{0,1\}$ be a function and $\rho \in \{0,1,\star\}^n$ be a partial assignment. We use $f|_\rho : \{0,1\}^n \to \{0,1\}$ to denote the restriction of f on ρ. That is, $f|_\rho(y) := f(\rho \circ y)$. $\mathrm{AC}^0_d[s(n)]$ denotes the family of Boolean circuits of size at most $s(n)$ and depth at most d. Such circuits can have AND, OR, NOT gates. Here, NOT gates do not count in the depth, and AND, OR gates can have unbounded fan-in. We measure the size of a circuit by the total number of wires (including input wires) in it. For the case of $d = 2$, we also use the terms DNF and CNF to refer to $\mathrm{OR} \circ \mathrm{AND}$ and $\mathrm{AND} \circ \mathrm{OR}$ circuits respectively. The width of a DNF or CNF is defined as the maximum of its bottom fan-in. We also use k-DNF (resp. k-CNF) to denote DNF (resp. CNF) of width at most k. A decision tree T is a binary tree. Each inner node of T is labelled with an index $i \in [n]$, and has exactly two children, which are labelled with 0 and 1. Each leaf of T is labelled with a Boolean value $b \in \{0,1\}$. A decision tree T computes a function in the following manner: on an input $x \in \{0,1\}^n$, we start from the root of T. In each turn we observe the index i of current node, query x_i and move to the left/right child depending on the bit x_i we received. Once we reach a leaf with label b, we output $T(x) = b$. The depth of a decision tree is the length of the longest path from root to any leaf. We also consider a special decision tree model for a list of functions.

Definition 4. Let $t, w, n, m \geq 1$ be integers. Let $\mathcal{F} = \{F_1, \ldots, F_m\}$ be a list of functions mapping from $\{0,1\}^n$ to $\{0,1\}$. A w-partial depth-t decision tree for $\mathcal{F}$ is a depth-t decision tree T satisfying the following. For every $F_i \in \mathcal{F}$ and every leaf ℓ of T, let $\alpha \in \{0,1,\star\}^n$ be the partial assignment that corresponds to ℓ (That is, α_i equals $\star$ if T does not query x_i before reaching ℓ, otherwise α_i equals to the value that leads T to move towards ℓ). Then it holds that $\mathrm{DT}(F_i|_\alpha) \leq w$.

Definition 5. Let n, m be two integers. Let H be a distribution over hash functions mapping $\{0,1\}^n$ into $\{0,1\}^m$. We say that H is k-wise independent, if for any k input-output pairs $(x_1, y_1), \ldots, (x_k, y_k) \in \{0,1\}^n \times \{0,1\}^m$ where $x_1, \ldots, x_t$ are distinct, it holds that

$$\Pr_{h \sim H}[\forall i \in [k], h(x_i) = y_i] = 2^{-km}.$$

We have the following standard construction of bounded independence hash functions (check e.g., [28, Chapter 3.5.5]).

Lemma 6. For every $n, m, k \geq 1$, there is an explicit k-wise independent hash functions H that maps $\{0,1\}^n$ into $\{0,1\}^m$. One can sample a function in H using $O(k(n+m))$ random bits. We also consider a weaker notion of pseudorandomness called "k-wise p-boundedness", first defined and studied by [18].

Definition 7. Suppose Λ is a random subset of $[n]$. We say that Λ is k-wise p-bounded if for any set $B \subseteq [n]$ of size at most k, it holds that $\Pr_\Lambda[B \subseteq \Lambda] \leq p^{|B|}$. For intuition, if we sample Λ by independently including each $i \in \Lambda$ with probability at most p, then Λ is n-wise p-bounded.

Theorem 8 ([5,26]). For every $m, n \in \mathbb{N}$ such that $m \geq n$ and every $\epsilon > 0$, there is an ϵ-PRG for $\mathrm{AC}^0_2[m]$ circuits (namely, CNF/DNF formulae) with seed length $O(\log(m)\log(m/\epsilon)\log\log(m))$.

Formal author main result

Theorem 1. For every $d \in \mathbb{N}$ the following is true. For every $m, n \in \mathbb{N}$ such that $m \geq n$ and every $\epsilon > 0$, there is an ϵ-PRG for $\mathrm{AC}^0_d[m]$ circuits with seed length $O(\log^{d-1}(m)\log(m/\epsilon)\log\log(m))$.

Author Section 4: improved PRG for constant-depth circuits

We give the formal statement of the derandomized multi-switching lemma below. This is the full version of Lemma 3 in the introduction.

Lemma 9. Let $t, w, k, n, m \geq 1$ be integers. Let $p, \epsilon > 0$ be reals. Let $\mathcal{F} = \{F_1, \ldots, F_m\}$ be a list of size-m k-DNFs on inputs $\{0,1\}^n$. Let $(\boldsymbol{\Lambda}, \boldsymbol{x})$ be a joint random variable satisfying the following:

- $\boldsymbol{\Lambda}$ is a $(t+k)$-wise p-bounded subset of $[n]$.
- Conditioning on $\boldsymbol{\Lambda}$, $\boldsymbol{x}$ is an n-bit random string that ϵ-fools size-(m^2) CNF.

Then we have

$$\Pr_{T \sim \boldsymbol{\Lambda}, x \sim \boldsymbol{x}}[\mathcal{F}|_{\Lambda[x,\star]} \text{ does not have } w\text{-partial depth-}t \text{ DT}] \leq 4m^{t/w}(24pk)^t + (24m)^{t+k}\epsilon.$$

We defer the proof of Lemma 9 to Section 6. Assuming Lemma 9, we prove Theorem 1. We prove a slightly stronger form of Theorem 1, stated below.

Theorem 10. For every $d \geq 2$ the following is true. For every $m, n \in \mathbb{N}$ such that $m \geq n$, $k \in \mathbb{N}$ and every $\epsilon > 0$, there is an ϵ-error, $O((\log^2(m) + k\log^{d-2}(m))\log(m/\epsilon)\log\log(m))$-seed PRG for $\mathrm{AC}^0_d[m]$ circuits with bottom fan-in bounded by k. Theorem 10 shows that, when the bottom fan-in of the AC^0_d circuits is smaller than $o(\log(m))$, we can hope for shorter seed length through our construction. Note that we can always interpret a depth-d AC^0 circuit with unbounded bottom fan-in as a depth-$(d+1)$ AC^0 circuit with bottom fan-in being 1. Then, Theorem 1 follows from Theorem 10 easily.

Proof. We use induction on the depth d. The case for $d = 2$ follows from Theorem 8. Assuming this is true for depth $d-1 \geq 2$, we prove it for the case of d. Let $w = 40k$ and $t = 80\log(m/\epsilon)$. We prepare the following pseudorandom primitives.

- First, let $H : [n] \to [w]$ be a $2t$-wise independent hash function, samplable using $O(\log(n)\log(m/\epsilon))$ bits. In the following, we will use H_i to denote $H^{-1}(i)$. We remark that H_i can be equivalently expressed as an n-bit string. Namely, $(H_i)_j = 1$ if and only if $j \in H_i$.
- Second, let $\epsilon' = \epsilon/(w2^{t+1})$. Sample $X_1, \ldots, X_w \in \{0,1\}^n$, each being an independent string that ϵ'-fools AC^0_{d-1}-circuits of size $2m^2$ and bottom-width $\log m$. When $d-1 \geq 3$, X_i is samplable using $O(\log^{d-2}(m)\log(m/\epsilon)\log\log(m))$ bits by the induction hypothesis. For the case $d-1 = 2$, X_i is samplable using $O(\log(m)\log(m/\epsilon)\log\log(m))$ bits by Theorem 8.
- Lastly, let $Y \in \{0,1\}^n$ be a random string that $\epsilon/(24m)^{2t}$-fools size-m CNF, samplable using $O(\log^2 m \log(m/\epsilon)\log\log m)$ bits (by Theorem 8).

The seed for our generator is the concatenation of the seeds used to sample all the primitives above. We compute the output of our generator as

$$Y \oplus (X_1 \wedge H_1) \oplus (X_2 \wedge H_2) \oplus \cdots \oplus (X_w \wedge H_w). \tag{1}$$

Here, $\wedge$ and $\oplus$ denote bit-wise AND and XOR respectively. The seed length is bounded by

$$\begin{aligned} &O(\log(n)\log(m/\epsilon) + w\log^{d-2}(m)\log(m2^t/\epsilon)\log\log(m) \\ &\quad + \log^2 m \log(m/\epsilon)\log\log m) \\ &\leq O((\log^2(m) + k\log^{d-2}(m))\log(m/\epsilon)\log\log(m)). \end{aligned}$$

We argue the correctness by a hybrid argument. Fix C to be an AC^0_d-circuit that we wish to fool. Let $U_1, \ldots, U_w$ denote w independent uniformly random strings from $\{0,1\}^n$. For every $i \in \{0,1,\ldots,w\}$, we define the i-th hybrid distribution as

$$\begin{aligned} D_i := Y \oplus (U_1 \wedge H_1) \oplus \cdots \oplus (U_i \wedge H_i) \\ \oplus (X_{i+1} \wedge H_{i+1}) \oplus \cdots \oplus (X_w \wedge H_w). \end{aligned} \tag{2}$$

We observe that D_0 is the output distribution of our PRG, while D_w is a uniformly random string from $\{0,1\}^n$. Hence, it suffices to show that

$$|\mathbb{E}_{x\sim D_0}[C(x)] - \mathbb{E}_{x\sim D_w}[C(x)]| \leq \epsilon. \tag{3}$$

To show (3), it suffices to show for every $i \in \{1,\ldots,w\}$ that

$$|\mathbb{E}_{x\sim D_{i-1}}[C(x)] - \mathbb{E}_{x\sim D_i}[C(x)]| \leq \epsilon/w. \tag{4}$$

In the following, we prove (4). We observe that H_i is $2t$-wise $1/w$-bounded. Conditioning on an instantiation of H, we have that $Z_i := Y \oplus \sum_{j<i}(U_j \wedge H_j) \oplus \sum_{j>i}(X_j \wedge H_j)$ is an $\epsilon/(24m)^{2t}$-pseudorandom string for size-(m^2) CNF, because Y is. Let $\mathcal{F}$ be the family of all next-to-bottom layer sub-circuits of C.

Denote by E the event "$\mathcal{F}|_{H_i[Z_i,\star]}$ does not have $\log(m)$-partial depth-t DT." Then it follows from Lemma 9 that

$$\Pr_{H,Y,U_1,\ldots,U_{i-1},X_{i+1},\ldots,X_w}[E] \leq 4m^{t/\log(m)}(24k/w)^t + \frac{\epsilon(24m)^{t+\log(m)}}{(24m)^{2t}} \leq \frac{\epsilon}{2w}.$$

Conditioning on $\neg E, H, Y, U_1, \ldots, U_{i-1}, X_{i+1}, \ldots, X_w$ and calculating Z_i as defined above, one can then write $C|_{H_i[Z_i,\star]}$ as a depth-t decision tree T where each leaf of T is labelled by an AC^0_{d-1}-circuit of size m^2 and bottom fan-in $\log(m)$. Let $\{\ell_1, \ldots, \ell_{2^t}\}$ enumerate the leaves of the decision tree. Each ℓ_j is associated with a size-m^2 depth-$(d-1)$ circuit, which is also denoted by $\ell_j : \{0,1\}^n \to \{0,1\}$ for brevity. Then one can write $C|_{(H_i)[Z_i,\star]}$ as

$$C|_{H_i[Z_i,\star]}(y) = \sum_{j=1}^{2^t} \ell_j(y)\mathbf{1}_{\{T(y) \text{ reaches leaf } \ell_j\}}.$$

Let's fix an index $j \in [2^t]$ for now. Note that $\ell_j(y)\mathbf{1}_{\{\text{reach } \ell_j \text{ on } y\}}$ is itself a depth-$(d-1)$ circuit of size at most $2m^2$. By the construction of X_i we know that

$$\mathbb{E}_{X_i}[\ell_j(X_i+Z_i)\mathbf{1}_{\{T \text{ reaches } \ell_j \text{ on } X_i+Z_i\}}] - \mathbb{E}_{U_i}[\ell_j(U_i+Z_i)\mathbf{1}_{\{T \text{ reaches } \ell_j \text{ on } U_i+Z_i\}}] \leq \frac{\epsilon}{2^{t+1}w}.$$

Taking a summation over all leaves j, one gets

$$\mathbb{E}_{X_i}[C|_{H_i[Z_i,\star]}(X_i+Z_i)] - \mathbb{E}_{U_i}[C|_{H_i[Z_i,\star]}(U_i+Z_i)]] \leq \epsilon/(2w).$$

Finally, one has

$$|\mathbb{E}_{x\sim D_{i-1}}[C(x)] - \mathbb{E}_{x\sim D_i}[C(x)]| \leq \Pr[\neg E]\frac{\epsilon}{2w} + \Pr[E] \leq \frac{\epsilon}{w},$$

proving (4). □

Given Theorem 10, we prove Theorem 1 by tuning parameters.

Proof of Theorem 1. For every $d \geq 3$ and every $\mathrm{AC}^0_d[m]$ circuit with unbounded bottom fan-in, we can interpret it as a depth-$(d+1)$ circuit with bottom fan-in being 1. Applying Theorem 10 in this case gives a PRG with seed length $O(\log^{d-1}(m)\log(m/\epsilon)\log\log(m))$, as desired. For the case of $d = 2$, we use Theorem 8 directly. This completes the proof. □As a final remark, suppose we could have $O(\log(m)\log(m/\epsilon))$-seed PRG for CNFs (namely, if we can shave the $\log\log m$ factor in Theorem 8). Then our construction implies PRG for AC^0_3 with seed length $O(\log^2(m)\log(m/\epsilon))$, and further implies PRG for AC^0_d with seed length $O(\log^{d-1}(m)\log(m/\epsilon))$, matching the lower bound barrier [14,13].

Author Section 5: fully-derandomized switching lemma

Before we show the proof of Lemma 9, we state and prove the simpler version of the classical switching lemma in this section. The following statement is the full version of Lemma 2 in Introduction. Lemma 2 follows from Lemma 11 trivially.

Lemma 11. Let $k, m \geq 1$ be integers, and $\epsilon, p > 0$ be reals. Let $F = \bigvee_{i=1}^{m} C_i$ be a size-m k-DNF over inputs $\{0,1\}^n$. For all $t \geq 1$, let $(\boldsymbol{\Lambda}, \boldsymbol{x})$ be a joint random variable such that:

- $\boldsymbol{\Lambda}$ is a $(t+k)$-wise p-bounded subset of $[n]$.
- Conditioning on $\boldsymbol{\Lambda}$, $\boldsymbol{x}$ is a random string that ϵ-fools CNF of size at most m.

Consider the random restriction $F|_{\Lambda[x,\star]}$, we have:

$$\Pr_{\Lambda,x}[\mathrm{DT}(F|_{\Lambda[x,\star]}) > t] \leq (10kp)^t + (4m)^{t+k}\epsilon.$$

Understanding the proof of Lemma 11 is necessary to read the proof of Lemma 9. On the other hand, once Lemma 11 is established, we can prove Lemma 9 using a rather similar strategy. The rest of the section is devoted to the proof of Lemma 11. We will first introduce the important concept of "canonical decision tree" ([14,27,25,26]) in Section 5.1. We prove for the case that x is truly random (and only Λ is pseudorandom) in Section 5.2, and then argue how to prove for pseudorandom x in Section 5.3. Throughout the whole section, we always use $F = \bigvee_{i=1}^{m} C_i$ to denote the k-DNF that we are analyzing. For every $i \in [m]$ let $V_i \subseteq [n]$ be the variables involved in the term C_i.

5.1 Canonical decision tree

For a DNF $F = \bigvee_{i=1}^{m} C_i$, denote by T_F the canonical decision tree of F, whose construction is shown in Algorithm 1. We use $\mathrm{CDT}(F)$ to denote the depth of the canonical decision tree for F. Since canonical decision tree is one particular decision tree of F, its depth must be no less than $\mathrm{DT}(F)$. To prove Lemma 11, we will analyze canonical decision trees, and show that with high probability over the random restriction ρ, we have $\mathrm{CDT}(F|_\rho) < t$.

Algorithm 1: Canonical Decision Tree. Input: A DNF $F = \bigvee_{i=1}^{m} C_i$, black-box access to a string $\alpha \in \{0,1\}^n$.

initialize: $j^* \leftarrow 0$; $x \leftarrow (\star)^n$.
while $j^* < m$ **do**
  Find the first $j > j^*$ such that $C_j(x) \not\equiv 0$. If no such j exists, exit the loop.
  $B_j \leftarrow$ the set of unknown variables in C_j.
  Query α_{B_j}.
  Set $x_{B_j} \leftarrow \alpha_{B_j}$.
  if $C_j(x) = 1$ **then return** 1.
  $j^* \leftarrow j$.
return 0.

5.2 Proof when x is truly random

Defining witness. Let $\rho = \Lambda[x, \star]$ being a bad restriction, under which $\mathrm{CDT}(F|_\rho) \geq t$. Consider simulating the canonical decision tree $T_{F|_\rho}$. We know that on some inputs $\alpha \in \{0,1\}^n$, $T_{F|_\rho}$ fails to output the decision after making $(t-1)$ queries. We choose one such α and simulate $T_{F|_\rho}$ until it makes at least t queries. The "running transcript" of $T_{F|_\rho}$ on α is naturally a witness to the fact that $\mathrm{CDT}(F_\rho) \geq t$. We formalize this idea in the following definition.

Definition 12. Let $F = \bigvee_{i=1}^m C_i$ be the k-DNF and $\rho \in \{0,1,\star\}^n$ be a restriction. Let $t \geq 1$. Consider a tuple $(r, \ell_i, s_i, B_i, \alpha_i)$, where:

1. $r \in [1, t]$ is an integer.
2. $(\ell_1, \ldots, \ell_r) \in [m]^r$ is a list of increasing indices.
3. $(s_1, \ldots, s_r)$ is a list of positive integers such that $s := \sum_{i=1}^r s_i \in [t, t+k-1]$.
4. $(B_1, \ldots, B_r)$ is a list of subsets of $[k]$. Moreover, for every $i \in [r]$, $|B_i| = s_i$.
5. $(\alpha_1, \ldots, \alpha_r)$ is a list of binary strings. For every $i \in [r]$, $|\alpha_i| = s_i$.

We call $(r, \ell_i, s_i, B_i, \alpha_i)$ a t-witness for ρ, if there exists $\alpha \in \{0,1\}^n$ such that:

- When we run $T_{F|_\rho}$ on α, for every $i \in [r]$, C_{ℓ_i} is the i-th term queried by $T_{F|_\rho}$.
- By the time $T_{F|_\rho}$ issues the i-th set of query, exactly s_i variables in C_{ℓ_i} are not known, and their "relative positions" in V_{ℓ_i} are specified by B_i.
- The response to the i-th set of query is α_i.

We define the size of the witness $(r, \ell_i, s_i, B_i, \alpha_i)$ as $s := \sum_{i=1}^r s_i$.

Notation convention. When B_i and α_i are associated with a term C_{ℓ_i}, sometimes we will slightly abuse notation by using B_i to refer to the set of variables it corresponds to in V_{ℓ_i} (Recall V_{ℓ_i} is the set of variables that appear in C_{ℓ_i}), and use α_i to denote a partial assignment $\alpha_i \in \{0,1,\star\}^n$ in which we only assign the B_i part of α_i and leave other coordinates unfixed. In this section, we will mostly use the term "witness" to denote t-witness, since we are always analyzing for a fixed t. It is easy to see that if $\mathrm{DT}(F|_\rho) \geq t$, there must be a witness for $\mathrm{CDT}(F|_\rho) \geq t$.[1] By now, a natural idea to bound the probability of picking a bad restriction would be (1) enumerating every possible witness, (2) calculating the probability of a random ρ having such a witness, and (3) union-bounding over them. Unfortunately this is too expensive for us: we have at least $\binom{m}{t}$ choices of the list (ℓ_i). In order for this approach to be meaningful, we have to bound the probability that a random restriction has a particular witness by m^{-t}, which seems very hard, if not impossible. It turns out we can avoid the enumeration of (ℓ_i) part in the witness. To succinctly describe the idea, let us define partial witnesses first.

[1] However, we note that the converse may not be true, since our witness can only refute the existence of shallow canonical decision trees.

Definition 13. Let $F = \bigvee_{i=1}^{m} C_i$ be the k-DNF and $\rho \in \{0,1,\star\}^n$ be a restriction. Let $t \geq 1$. Consider a tuple (r, s_i, B_i, α_i). We call (r, s_i, B_i, α_i) a partial t-witness for ρ, if there exists $(\ell_1, \ldots, \ell_r)$ such that $(r, \ell_i, s_i, B_i, \alpha_i)$ is a t-witness for ρ.

Remark 14. Here we make an important observation: if (r, s_i, B_i, α_i) is a partial witness for ρ, then there is only one valid list (ℓ_i) which makes $(r, \ell_i, s_i, B_i, \alpha_i)$ a witness for ρ (To see this, first note that ℓ_1 is fixed. After querying ℓ_1 and getting the response α_1, use induction). This observation will prove useful in Section 5.3 and Section 6. Then, the proof goes by enumerating r, s_i, B_i, α_i and bounding the following:

$$\Pr_{\rho}[\mathrm{DT}(F|_\rho) \geq t] \leq \sum_{(r,s_i,B_i,\alpha_i)} \Pr_{\rho}[(r, s_i, B_i, \alpha_i) \text{ is a partial witness for } \rho]. \quad (5)$$

Fixing an $s \in [t, t+k-1]$, there are at most $(4k)^s$ possible partial witnesses (r, s_i, B_i, α_i) of size s. For each of them, if we can bound the probability by, say, $(2p)^s$, then the lemma is proved.

The witness searcher. Now, given a restriction ρ and a candidate partial witness (r, s_i, B_i, α_i), how can we decide if there is a list $(\ell_i)_i$ such that $(r, \ell_i, s_i, B_i, \alpha_i)$ constitutes a witness for ρ? We will do so by designing and using a simple procedure, which we call the witness searcher. More specifically, our searcher receives as input a restriction ρ, a partial witness (r, s_i, B_i, α_i) and an advice string $y \in \{0,1\}^n$. It outputs either a complete witness $(r, \ell_i, s_i, B_i, \alpha_i)$, or a special ERROR symbol. Its description in shown in Algorithm 2. In the following, we use S to denote Algorithm 2.

Algorithm 2: Witness Searcher. Input: A DNF $F = \bigvee_{i=1}^{m} C_i$, a restriction $\rho \in \{0,1,\star\}^n$, a partial witness (r, s_i, B_i, α_i), an advice $y \in \{0,1\}^n$.

initialize: $z \leftarrow \rho \circ y$; $j^* \leftarrow 0$; $c \leftarrow 1$.
while $c \leq r$ **do**
  Find the first $j > j^*$ such that C_j is satisfied by z. If no such i exists, return ERROR.
  Set $\ell_c \leftarrow j$, and associate B_c, α_c with C_{ℓ_c}.
  Replace the B_c part of z with α_c. That is, $z \leftarrow \rho \circ \alpha_1 \circ \cdots \circ \alpha_c \circ y$.
  $c \leftarrow c+1$; $j^* \leftarrow j$.
return $(r, \ell_i, s_i, B_i, \alpha_i)$.

Lemma 15. If (r, s_i, b_i, α_i) is a partial witness for ρ, then there exists an advice y that makes S find $(\ell_i)_{i=1}^{r}$. More importantly, on a uniformly random $y \sim U_n$, S finds $(\ell_1, \ldots, \ell_r)$ with probability exactly 2^{-s}.

Proof. If $(r, s_i, \ell_i, b_i, \alpha_i)$ is a witness for ρ, it records a running transcript of $T_{F|_\rho}$ on some input $\alpha \in \{0,1\}^n$. This mean that C_{ℓ_1} is the first term queried by $T_{F|_\rho}$, which implies:

1. Every term before C_{ℓ_1} is falsified by ρ (because $T_{F|_\rho}$ skipped them).
2. C_{ℓ_1} is consistent with ρ (because $T_{F|_\rho}$ was uncertain of the value of $C_{\ell_i}(\alpha)$.

Now, consider running S on (r, s_i, b_i, α_i), S will also skip all the terms before C_{ℓ_1}. At C_{ℓ_1}, since ρ is consistent with C_{ℓ_1}, the random string $\rho \circ y$ satisfies C_{ℓ_1} with probability $2^{-|B_1|}$. Conditioning on this happened, in the later execution, we modify z and replace the part corresponding to B_1 with α_1. At this point, we go back and inspect the execution of $T_{F|_\rho}$ on α. Since $(r, s_i, \ell_i, b_i, \alpha_i)$ is the running transcript of $T_{F|_\rho}$ on α, we know that α_1 was the response that $T_{F|_\rho}$ received from querying alive variables in C_{ℓ_1}. Since $T_{F|_\rho}$ issued its second bunch of queries to alive variables in C_{ℓ_2}, it implies that terms from C_{ℓ_1+1} to C_{ℓ_2-1} are all falsified by $\rho \circ \alpha_1$, and C_{ℓ_2} is consistent with $\rho \circ \alpha_1$. Then, it follows that $\rho \circ \alpha_1 \circ y$ satisfies C_{ℓ_2} with probability $2^{-|B_2|}$. Conditioning on this happened, the searcher will proceed with string $\rho \circ \alpha_1 \circ \alpha_2 \circ y$ and we can again consider the execution of $T_{F|_\rho}$ after querying C_{ℓ_2}. We do this argument so on and so forth, until we have identified all of r indices $\ell_1, \ldots, \ell_r$ and exit the procedure. In summary, we have shown there exists y which makes S find the list $(\ell_i)_{i=1}^r$, and the probability of sampling such a y is $2^{-\sum_{i=1}^c |B_i|} = 2^{-s}$ as desired. □

Decoupling. If we inspect the execution of S carefully, we can notice that it only needs to know the string $z = \rho \circ y$ to work. In particular, it does not need to know which part of z is fixed in the restriction ρ. Therefore, we can revise S to get a searcher S': the input to S' is now a string z and (r, s_i, B_i, α_i). Otherwise it runs identically the same way as S. We denote the output of S' as $S'(z, (r, s_i, B_i, \alpha_i))$. By Lemma 15, we have

$$\begin{aligned}&\Pr_{\rho\sim R}[(r, s_i, B_i, \alpha_i) \text{ is a partial witness for } \rho] \\ &\qquad\qquad \le 2^s \Pr_{\rho\sim R, y\sim U_n}[S'(\rho \circ y, (r, s_i, B_i, \alpha_i)) \text{ is a witness for } \rho].\end{aligned}$$

Denote $\rho = \Lambda[x, \star]$. We observe that

$$\mathbf{1}_{\{(r,\ell_i,s_i,B_i,\alpha_i) \text{ is a witness for } \rho\}} \le \mathbf{1}_{\{(\bigcup_{j=1}^r B_j)\subseteq\Lambda\}}. \tag{6}$$

For a $(t+k)$-wise p-bounded set Λ, the event on the right hand side holds with probability $p^{\sum_j |B_j|}$. Then, we have

$$\begin{aligned}&\Pr_{\rho\sim\Lambda[x,0], y\sim U_n}[S'(\rho \circ y, (r, s_i, B_i, \alpha_i)) \text{ is a witness for } \rho] \\ &= \mathbb{E}_\Lambda \mathbb{E}_{x,y\sim U_n}[\mathbf{1}_{\{S'(\Lambda[x,y],(r,s_i,B_i,\alpha_i)) \text{ is a witness for } \rho\}}] \\ &= \mathbb{E}_{z\sim U_n}\left[\Pr_\Lambda[S'(z, (r, s_i, B_i, \alpha_i)) \text{ is a witness for } \rho]\right] \\ &\le p^s,\end{aligned}$$

where the third equality is due to that $\Lambda[x, y]$ is distributed as U_n when $x, y \sim U_n$, and the last inequality holds by (6) and the $(t+k)$-wise p-bounded property of Λ.

Wrapping-up. We finish the proof by enumerating all (r, s_i, B_i, α_i) and taking a summation.

$$\begin{aligned}
&\Pr_{\rho\sim\Lambda[x,\star]}[\mathrm{DT}(F|_\rho) \geq t] \\
&\leq \sum_{(r,s_i,B_i,\alpha_i)} \Pr_{\rho\sim\Lambda[x,\star]}[(r, s_i, B_i, \alpha_i) \text{ is a partial witness for } \rho] \\
&\leq \sum_{(r,s_i,B_i,\alpha_i)} 2^s \mathbb{E}_{z\in U_n}\left[\Pr_{\Lambda}[S'(z, (r, s_i, B_i, \alpha_i)) \text{ is a witness for } \Lambda[x,\star]]\right] \\
&\leq \sum_{s=t}^{t+k} 2^s (4k)^s p^s \\
&\leq (10kp)^t. \qquad (7)
\end{aligned}$$

5.3 Proof when x is pseudorandom

Now we consider the case that x is not truly random. There could even be correlation between x and Λ. Our only requirement for x is that, for every fixed Λ, x is a pseudorandom string that ϵ-fools CNFs of size at most m. First of all, by Remark 14, the following equation is established.

$$\mathbf{1}_{\{(r,s_i,B_i,\alpha_i) \text{ is a partial witness for } \rho\}} = \sum_{\ell_i} \mathbf{1}_{\{(r,\ell_i,s_i,B_i,\alpha_i) \text{ is a witness for } \rho\}}. \qquad (8)$$

If we inspect the deduction in (7), it actually shows the following:

$$\sum_{(r,s_i,B_i,\alpha_i)} \sum_{\ell_i} \Pr_{\rho\sim\Lambda[U_n,\star]}[(r, \ell_i, s_i, B_i, \alpha_i) \text{ is a witness for } \rho] \leq (10kp)^t. \qquad (9)$$

Next, fixing the set Λ and a tuple of $(r, \ell_i, s_i, B_i, \alpha_i)$, we consider the predicate

$$h_\Lambda^{(r,\ell_i,s_i,B_i,\alpha_i)}(x) := \mathbf{1}_{\{(r,\ell_i,s_i,B_i,\alpha_i) \text{ is a witness for } \Lambda[x,\star]\}}.$$

We omit the superscript and subscript of h when they are clear from context. We claim that h can be implemented by a CNF of size at most m. To see this, note that $h(x) = 1$ if all of the following hold.

- For every $j < \ell_1$, C_j is falsified by $\Lambda[x, \star]$. For this to be true, there is at least one variable in C_j that is not chosen by Λ and is assigned to the opposite value to its appearance in C_j. Note that we can use an OR over relevant variables or their negations to verify if C_j is falsified.
- For $j = \ell_1$, C_{ℓ_1} cannot be falsified by $\Lambda[x, \star]$, which means all variables in C_{ℓ_1} that are not chosen by Λ are assigned so that it does not contradict C_{ℓ_1}. We can use an AND over those variables to verify this.
- For every $\ell_1 < j < \ell_2$, C_j is falsified by $\Lambda[x, \alpha_1]$, which means that either C_j is falsified by α_1, or there is at least one variable in C_j that is not chosen by Λ and is assigned to the opposite value. If C_j is falsified by α_1, there is nothing to verify. Otherwise, we can verify it by an OR over relevant variables.

- The same reasoning applies to C_j for every $j \geq \ell_2$. Finally, after we finish the verification for C_{ℓ_r}, we are done.

In summary, we can implement h by a CNF (AND of ORs). It is obvious that the size of the CNF is no larger than the size of F, which is m. Therefore, we conclude that x ϵ-fools h. That is,

$$\Pr_{x\sim U_n}[h_\Lambda^{(r,\ell_i,s_i,B_i,\alpha_i)}(x)] - \Pr_{x\sim \boldsymbol{x}}[h_\Lambda^{(r,\ell_i,s_i,B_i,\alpha_i)}(x)] \leq \epsilon. \tag{10}$$

Finally, we have

$$\begin{aligned}
&\Pr_{\rho\sim\Lambda[\boldsymbol{x},\star]}[\mathrm{DT}(F|_\rho) \geq t] \\
&\leq \sum_{(r,s_i,B_i,\alpha_i)} \Pr_{\rho\sim\Lambda[\boldsymbol{x},\star]}[(r,s_i,B_i,\alpha_i) \text{ is a partial witness for } \rho] \\
&\leq \sum_{(r,s_i,B_i,\alpha_i)} \sum_{\ell_i} \Pr_{\rho\sim\Lambda[\boldsymbol{x},\star]}[(r,\ell_i,s_i,B_i,\alpha_i) \text{ is a witness for } \rho] \\
&\leq \sum_{(r,s_i,B_i,\alpha_i)} \sum_{\ell_i} \epsilon + \Pr_{\rho\sim\Lambda[U_n,\star]}[(r,\ell_i,s_i,B_i,\alpha_i) \text{ is a witness for } \rho] \\
&\leq (10kp)^t + (4m)^{t+k}\epsilon.
\end{aligned}$$

Here, the first line is due to the argument in Section 5.2. The second line holds by (8). The third line is due to (10). The last line holds because (9) and there are at most $(4m)^{k+t}$ possible tuples $(r,\ell_i,s_i,B_i,\alpha_i)$.

Author Section 6: derandomizing the multi-switching lemma

In this section, we prove Lemma 9. The rest of the section is devoted to the proof of Lemma 9. Section 6.1 states some preliminary tools and includes a proof overview. Section 6.2 considers the case that x is truly random. It extends the idea in Section 5.2 and has a rather similar structure. Finally in Section 6.3, we consider the case that x is pseudorandom, and finish the proof.

6.1 Preliminaries

The canonical partial decision tree. Let ρ be a random restriction under which $\mathcal{F}|_\rho$ fails to have a w-partial depth-t decision tree. Let $\beta \in \{0,1\}^n$ be an unknown string that our decision tree shall query. We consider the following attempt (see Algorithm 3) to construct a partial decision tree for $\mathcal{F}|_\rho$. Here $z \in \{0,1\}^n$ is an auxiliary string to be chosen. Every choice of $z \in \{0,1\}^n$ yields a different partial decision tree for $\mathcal{F}$. In [25], the construction was called "canonical partial decision tree".

Algorithm 3: Canonical Partial Decision Tree. Input: A list of DNFs $\mathcal{F} = \{F_1,\ldots,F_m\}$, black-box access to a string $\beta \in \{0,1\}^n$, and an auxiliary string $z \in \{0,1\}^n$.

initialize: $x \leftarrow (\star)^n$; $j \leftarrow 1$; counter $\leftarrow 0$.
while counter $< t$ **do**
  Find the smallest $i \geq j$ such that $\mathrm{DT}(F_i|_x) > w$. If no such i exists, exit the loop.
  $y \leftarrow (\star)^n$; $I \leftarrow \emptyset$.
  while $F_i|_{x \circ y}(\star)$ is not constant and counter $< t$ **do**
    $C_{i,q} \leftarrow$ the term that $T_{F_i|_{x \circ y}}$ will query.
    $B_{i,q} \leftarrow$ the set of unknown variables in $C_{i,q}|_{x \circ y}$.
    $y_{B_{i,q}} \leftarrow z_{B_{i,q}}$.
    $I \leftarrow I \cup B_{i,q}$.
    counter $\leftarrow$ counter $+ |B_{i,q}|$.
  Query β_I, and set $x_I \leftarrow \beta_I$.
  $j \leftarrow i$.
return x.

Note that in Algorithm 3, it is possible that for a formula F_i to get picked by the outer "while" loop more than once.

Proof overview. Before we dive into the formal proof, we try to give some (over-simplified) intuition here. If $\mathcal{F}$ does not have w-partial DT of depth t, then Algorithm 3 fails to construct a partial decision tree of depth t on every auxiliary string z. However, we will only consider some "adversarially chosen" z's. That is, we only consider those z's that trick Algorithm 3 to make at least w queries on each chosen formula. By doing so, we are guaranteed that Algorithm 3 only chooses t/w formulae in total. Then, we can pay a factor of $m^{t/w}$ to enumerate a subset of t/w formulae in $\mathcal{F}$, and calculate the probability that Algorithm 3 "gets stuck" on those formulae. Having fixed those t/w formulae, we can use ideas similar to Section 5.2, and bound the probability by $O(pk)^t$. In summary, we get $\Pr[\text{bad restriction}] \leq m^{t/w} O(pk)^t$. Here we only considered the case that the restriction string x is purely random. For the case that both Λ and x are pseudorandom, we use tricks similar to Section 5.3 and pay another additive factor.

6.2 Proof when x is truly random

We start the proof by considering the case that x is a truly random string. Fix ρ to be a bad restriction, under which $\mathcal{F}|_\rho$ fails to have a w-partial depth-t decision tree. It implies that on every $z \in \{0,1\}^n$, Algorithm 3 fails to construct a partial decision tree for $\mathcal{F}$. However, for ease of our analysis, we will only consider a special class of string z. We give the following definition.

Definition 16. Let ρ be a bad restriction for $\mathcal{F}$. We call $z \in \{0,1\}^n$ a powerful (w,t)-refutation for ρ, if there exists $\beta \in \{0,1\}^n$ satisfying the following: Algorithm 3 on input $(\mathcal{F}|_\rho, \beta, z)$ makes at least t queries. Moreover, at each time we query β_I, it is guaranteed that $|I| \geq w$. Powerful refutations always exist for bad restrictions, as shown in the following.

Lemma 17. If $\mathcal{F}|_\rho$ does not have w-partial decision tree of depth t, then there exists a powerful (w,t)-refutation for ρ.

Proof. If $\mathcal{F}_\rho$ does not have w-partial decision tree of depth t, then it means Algorithm 3 fails to construct a w-partial DT of depth t for every $z \in \{0,1\}^n$. Then we construct a powerful refutation z, together with its associated adversarial input β, in the following way.

- Initially, we set $z = \beta = (\star)^n$. Then we monitor the execution of Algorithm 3 on $(\mathcal{F}, \beta, z)$, and gradually fill in z, β when they are accessed by Algorithm 3.
- Whenever Algorithm 3 chooses one formula $F_i|_{\rho \circ x}$ in the outer while-loop, we do the following:
  - When running inside the inner while-loop, Algorithm 3 needs to consult the auxiliary string z. We can fill in relevant variables of z in such a way that Algorithm 3 consults at least w bits from z in the loop. Since $\mathrm{DT}(F_i|_{\rho \circ x}) \geq w$, this is always possible.
  - After finishing the inner loop, Algorithm 3 will make a query to β_I. At this point, we set β_I in such a way that $\mathcal{F}|_{\rho \circ x \circ \beta}$ does not have w-partial DT of depth $(t - \text{counter})$. This is always possible as long as $\mathcal{F}_{\rho \circ x}$ does not have w-partial DT of depth $(t - \text{counter} + |I|)$, which holds by induction.
- After Algorithm 3 returns, we fill in the remaining bits of β, z with zeros.

□

Having established Lemma 17, we come up with the following definition of "global witness" naturally.

Definition 18. Let t, w be two integers. Consider a list of k-DNFs $\mathcal{F} = \{F_1, \ldots, F_m\}$. Suppose $\rho \in \{0,1,\star\}^n$ is a restriction. Let $(R, L_i, S_i, W_i, \beta_i)$ be a tuple, where

1. $1 \leq R \leq t/w$ is an integer;
2. $1 \leq L_1 \leq L_2 \leq \cdots \leq L_R \leq m$ is a list of R non-decreasing indices;
3. $S_1, \ldots, S_R$ is a list of R integers such that $\sum_{i=1}^R S_i \in [t, t+k]$;
4. $W_1, \ldots, W_R$ is a list of witnesses (as per Definition 12). For every $i \in [R]$, W_i has size S_i;
5. $\beta_1, \ldots, \beta_R$ are R strings where $|\beta_i| = S_i$ for every $i \in [R]$.

We call the tuple a (w,t)-global witness for ρ, if it satisfies the following.

1. Set $\rho_1 = \rho$. W_1 is a S_1-witness for $F_{L_1}|_{\rho_1}$.
2. For every $i \geq 2$, let $I_{i-1} \subseteq [n]$ be the set of variables involved in W_{i-1}. Note that $|I_{i-1}| = S_{i-1}$ since the size of W_{i-1} is S_{i-1}. Identify β_{i-1} as a partial assignment in $\{0,1,\star\}^n$ where only the part $\beta_{i-1,I_{i-1}}$ is set and other coordinates are filled in with $\star$. Construct $\rho_i = \rho_{i-1} \circ \beta_{i-1}$. Then W_i is a S_i-witness for $F_{L_i}|_{\rho_i}$.

The size of the global witness is defined as $\sum_{i=1}^R S_i$. As a corollary of Lemma 17, we have:

Corollary 19. Consider a list of k-DNFs $\mathcal{F} = \{F_1, \ldots, F_m\}$. Suppose $\rho \in \{0,1,\star\}^n$ is a restriction such that $\mathcal{F}|_\rho$ does not have w-partial depth-t decision tree. Then there exists a (w,t)-global witness for ρ.

Proof. By Lemma 17, there exists a powerful (w,t)-refutation for $\mathcal{F}|_\rho$. Take one such refutation z with its adversarial input β. Inspect the execution of Algorithm 3 on $(\mathcal{F}|_\rho, \beta, z)$ and record the transcript. The transcript contains the desired tuple. □

Similar to what we have done in Section 5.2, we define the global partial witness.

Definition 20. Let t, w be two integers. Consider a list of k-DNFs $\mathcal{F} = \{F_1, \ldots, F_m\}$. Suppose $\rho \in \{0,1,\star\}^n$ is a restriction. Let $(R, L_i, S_i, P_i, \beta_i)$ be a tuple, where

1. $1 \le R \le t/w$ is an integer;
2. $1 \le L_1 \le L_2 \le \cdots \le L_R \le m$ is a list of R non-decreasing indices;
3. $S_1, \ldots, S_R$ is a list of R integers such that $\sum_{i=1}^{R} S_i \in [t, t+k]$;
4. $P_1, \ldots, P_R$ is a list of partial witnesses. For every $i \in [R]$, P_i has size S_i.
5. $\beta_1, \ldots, \beta_R$ are R strings where $|\beta_i| = S_i$ for every $i \in [R]$.

We call $(R, L_i, S_i, P_i, \beta_i)$ a (w,t)-global partial witness for ρ, if we can complete P_i to get a witness W_i for every $i \in [R]$, such that $(R, L_i, S_i, W_i, \beta_i)$ is a global witness for ρ.

Remark 21. Again, it is easy to see by induction that, for every global partial witness, there is exactly one way to complete it and get a global witness. By a simple counting, it turns out for every $S \in [t, t+k]$, there are at most $2m^{t/w}(12k)^S$ possible global partial witnesses of size S:

1. First, $2m^{t/w}$ ways to choose at most t/w formulae in $\mathcal{F}$.
2. Then, 3^S ways to partition the s units of "query budget" into R partial witnesses as $S = \sum_{i=1}^{R} S_i$, and further partition the budget for each partial witness as $S_i = \sum_{j=1}^{r_i} s_{i,j}$.
3. Then, at most k^S ways to construct sets $B_{i,j}$ for each partial witness.
4. Finally, there are 4^S ways to choose β_i's, as well as $\alpha_{i,j}$'s in each partial witness.

Fixing a global partial witness $(R, L_i, S_i, P_i, \beta_i)$, we try to bound the probability that a random ρ has this witness by $(2p)^k$. Using the witness searcher (Algorithm 2) as a subroutine, we design a global witness searcher first. Again, the global witness searcher needs access to an advice string $y \in \{0,1\}^n$. See Algorithm 4. In the following, we use S to denote Algorithm 4 for brevity. Based on Lemma 15, we prove:

Lemma 22. If $(R, L_i, S_i, P_i, \beta_i)$ is indeed a global partial witness for ρ, then there exists an advice y that makes S find $(R, L_i, S_i, P_i, \beta_i)$. More importantly, on a uniformly random $y \sim U_n$, S finds $(\ell_1, \ldots, \ell_r)$ with probability exactly 2^{-S}, where $S := \sum_{i=1}^{R} S_i$ is the size of the global witness.

Algorithm 4: Global Witness Searcher. Input: A list of DNFs $\mathcal{F} = \{F_1, \ldots, F_m\}$, a restriction $\rho \in \{0, 1, \star\}^n$, a global partial witness $(R, L_i, S_i, P_i, \beta_i)$, and an advice $y \in \{0, 1\}^n$.

initialize: $z \leftarrow \rho \circ y$; $c \leftarrow 1$; $\rho^{(1)} \leftarrow \rho$.
while $c \leq R$ **do**
Run Algorithm 2 on $(F_{L_c}, \rho^{(c)}, P_c, y)$. If it reports ERROR, report ERROR and terminate the procedure. Otherwise let W_c be the witness returned.
$I_c \leftarrow$ the set of variables involved in W_c.
Identify β_c as a partial assignment, where only β_{I_c} is fixed.
$\rho^{(c+1)} \leftarrow \rho^{(c)} \circ \beta_c$.
$c \leftarrow c + 1$.
return x.

Proof. We examine the execution of S. It first runs Algorithm 2 as a subroutine, trying to find W_1 for P_1. With probability 2^{-S_1} over y, Algorithm 2 succeeds in finding W_1. Conditioning on this happened, we have only committed the assignment of y_{I_1}, and the rest part of y is still uniformly random. Also note that we set $\rho^{(2)}_{I_1}$ to β_1, which will hide y_{I_1} in the later execution. This enables us to do an induction and finish the proof. □

Decoupling. Now, we can observe that S only needs to know $z = \rho \circ y$ to work. In particular, it does not need to know which part of z is fixed in ρ. Therefore, we can revise S to get a searcher S': the input to S' is now a string z and a global partial witness $(R, L_i, S_i, P_i, \beta_i)$. Otherwise it runs identically the same way as S. We denote the output of S' as $S'(z, (R, L_i, S_i, P_i, \beta_i))$. By Lemma 22, we have

$$\Pr_{\rho \sim R}[(R, L_i, S_i, P_i, \beta_i) \text{ is a global partial witness for } \rho] \leq 2^S \Pr_{\rho \sim R, y \sim U_n}[S'(\rho \circ y, (R, L_i, S_i, P_i, \beta_i)) \text{ is a global witness for } \rho].$$

We observe that

$$\mathbf{1}_{\{(R, L_i, S_i, W_i, \beta_i) \text{ is a global witness for } \rho\}} \leq \mathbf{1}_{\{(\bigcup_{j=1}^{R} I_j) \subseteq \Lambda\}}. \tag{11}$$

For a $(t + k)$-wise p-bounded Λ, the event on the right hand side holds with probability $p^{\sum_j |I_j|}$. Then, we have

$$\begin{aligned}
&\Pr_{\rho \sim \Lambda[x,0], y \sim U_n}[S'(\rho \circ y, (R, L_i, S_i, P_i, \beta_i)) \text{ is a global witness for } \rho] \\
&= \mathbb{E}_{\Lambda} \mathbb{E}_{x, y \sim U_n}[\mathbf{1}_{\{S'(\Lambda[x,y], (R, L_i, S_i, P_i, \beta_i)) \text{ is a global witness for } \rho\}}] \\
&= \mathbb{E}_{z \sim U_n}\left[\Pr_{\Lambda}[S'(z, (R, L_i, S_i, P_i, \beta_i)) \text{ is a global witness for } \rho]\right] \\
&\leq p^S,
\end{aligned}$$

where the third line is due to that $\Lambda[x,y]$ is distributed as U_n when $x,y \sim U_n$, and the last inequality holds by (11) and the $(t+k)$-wise p-bounded property of Λ.

Wrapping-up. We finish the proof by enumerating all $(R, L_i, S_i, P_i, \beta_i)$ and taking a summation.

$$\begin{aligned}
&\Pr_{\rho\sim\Lambda[x,\star]}[\mathcal{F}|_\rho \text{ does not have } w\text{-partial depth-}t \text{ decision tree}] \\
&\le \sum_{(R,L_i,S_i,P_i,\beta_i)} \Pr_{\rho\sim\Lambda[x,\star]}[(R,L_i,S_i,P_i,\beta_i) \text{ is a global partial witness for } \rho] \\
&\le \sum_{(R,L_i,S_i,P_i,\beta_i)} 2^S \mathbb{E}_{z\in U_n}\left[\Pr_\Lambda[S'(z,(R,L_i,S_i,P_i,\beta_i)) \text{ is a global witness for } \Lambda[x,\star]]\right] \\
&\le \sum_{S=t}^{t+k} 2m^{t/w} 2^S (12k)^S p^S \\
&\le 4m^{t/w}(24kp)^t. \qquad (12)
\end{aligned}$$

6.3 Proof when x is pseudorandom

Now we consider the case that x is not truly random. Our only requirement for x is that, for every fixed Λ, x is a pseudorandom string that ϵ-fools CNFs of size at most m. First of all, by Remark 21, the following equation is established.

$$\begin{aligned}
&\mathbf{1}_{\{(R,L_i,S_i,P_i,\beta_i) \text{ is a global partial witness for } \rho\}} \\
&\qquad = \sum_{W_i:\text{completion of } P_i} \mathbf{1}_{\{(R,L_i,S_i,W_i,\beta_i) \text{ is a global witness for } \rho\}}. \qquad (13)
\end{aligned}$$

Then, the deduction in (12) implies that

$$\sum_{(R,L_i,S_i,W_i,\beta_i)} \Pr_{\rho\sim\Lambda[U_n,\star]}[(R,L_i,S_i,W_i,\beta_i) \text{ is a global witness for } \rho] \le 4m^{t/w}(24kp)^t. \qquad (14)$$

Next, fixing a tuple $(R, L_i, S_i, W_i, \beta_i)$ and an instantiation of Λ, we consider the predicate

$$h_\Lambda^{(R,L_i,S_i,W_i,\beta_i)}(x) := \mathbf{1}_{\{(R,L_i,S_i,W_i,\beta_i) \text{ is a global witness for } \Lambda[x,\star]\}}.$$

We omit the superscript and subscript of h when they are clear from context. We claim that h can be implemented by a CNF of size at most $\sum_{i=1}^m \text{size}(F_i) \le m^2$. To see this, note that $h(x) = 1$ if for every $i \in [R]$, W_i is a witness for $\rho \circ \beta_1 \circ \cdots \circ \beta_{i-1}$. By argument in Section 6.3, this can be verified using a CNF of size $\text{size}(F_{L_i})$. Therefore, we can evaluate h using an AND over R CNFs, which is itself a larger CNF. Therefore, we conclude that x ϵ-fools h. That is,

$$\Pr_{x\sim U_n}[h_\Lambda^{(R,L_i,S_i,W_i,\beta_i)}(x)] - \Pr_{x\sim \boldsymbol{x}}[h_\Lambda^{(R,L_i,S_i,W_i,\beta_i)}(x)] \le \epsilon. \qquad (15)$$

Finally, we have

$$\begin{aligned}
&\Pr_{\rho\sim\Lambda[\boldsymbol{x},\star]}[\mathcal{F}|_\rho \text{ does not have } w\text{-partial depth-}t \text{ decision tree}] \\
&\leq \sum_{(R,L_i,S_i,W_i,\beta_i)} \Pr_{\rho\sim\Lambda[\boldsymbol{x},\star]}[(R,L_i,S_i,W_i,\beta_i) \text{ is a global witness for } \rho] \\
&\leq \sum_{(R,L_i,S_i,W_i,\beta_i)} \epsilon + \Pr_{\rho\sim\Lambda[U_n,\star]}[(R,L_i,S_i,W_i,\beta_i) \text{ is a global witness for } \rho] \\
&\leq 4m^{t/w}(24kp)^t + (24m)^{t+k}\epsilon.
\end{aligned}$$

Here, the first line is due to the argument in Section 6.2 and observation (13). The second line is due to (15). The last line holds because (14) and there are at most $(24m)^{t+k}$ possible tuples $(R, L_i, S_i, W_i, \beta_i)$.

References

[1] Miklós Ajtai and Avi Wigderson. Deterministic simulation of probabilistic constant depth circuits. Adv. Comput. Res., 5:199–222, 1989.

[2] Paul Beame, Russell Impagliazzo, and Srikanth Srinivasan. Approximating ac^0 by small height decision trees and a deterministic algorithm for #ac^0sat. In Computational Complexity Conference, pages 117–125. IEEE Computer Society, 2012.

[3] Mark Braverman. Polylogarithmic independence fools AC 0 circuits. J. ACM, 57(5):28:1–28:10, 2010. doi:10.1145/1754399.1754401.

[4] Ruiwen Chen, Rahul Santhanam, and Srikanth Srinivasan. Average-case lower bounds and satisfiability algorithms for small threshold circuits. Theory Comput., 14(1):1–55, 2018. doi:10.4086/toc.2018.v014a009.

[5] Anindya De, Omid Etesami, Luca Trevisan, and Madhur Tulsiani. Improved pseudoran- dom generators for depth 2 circuits. In Maria J. Serna, Ronen Shaltiel, Klaus Jansen, and José D. P. Rolim, editors, Approximation, Randomization, and Combinatorial Optimiza- tion. Algorithms and Techniques, 13th International Workshop, APPROX 2010, and 14th International Workshop, RANDOM 2010, Barcelona, Spain, September 1-3, 2010. Proceed- ings, volume 6302 of Lecture Notes in Computer Science, pages 504–517. Springer, 2010. doi:10.1007/978-3-642-15369-3_38.

[6] Dean Doron, Raghu Meka, Omer Reingold, Avishay Tal, and Salil P. Vadhan. Monotone branching programs: Pseudorandomness and circuit complexity. Electron. Colloquium Comput. Complex., page 18, 2021. URL: https://eccc.weizmann.ac.il/report/2021/018.

[7] Michael A. Forbes and Zander Kelley. Pseudorandom generators for read-once branching pro- grams, in any order. In Mikkel Thorup, editor, 59th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2018, Paris, France, October 7-9, 2018, pages 946–955. IEEE Computer Society, 2018. doi:10.1109/FOCS.2018.00093.

[8] Oded Goldreich and Avi Wigderson. On the size of depth-three boolean circuits for computing multilinear functions. Electron. Colloquium Comput. Complex., page 43, 2013. URL: https://eccc.weizmann.ac.il/report/2013/043.

[9] Parikshit Gopalan, Raghu Meka, and Omer Reingold. DNF sparsification and a faster deterministic counting algorithm. Comput. Complex., 22(2):275–310, 2013. doi:10.1007/ s00037-013-0068-6.

[10] Parikshit Gopalan, Raghu Meka, Omer Reingold, Luca Trevisan, and Salil P. Vadhan. Better pseudorandom generators from milder pseudorandom restrictions. In 53rd Annual IEEE Symposium on Foundations of Computer Science, FOCS 2012, New Brunswick, NJ, USA, October 20-23, 2012, pages 120–129. IEEE Computer Society, 2012. doi:10.1109/FOCS.2012. 77.

[11] Elad Haramaty, Chin Ho Lee, and Emanuele Viola. Bounded independence plus noise fools products. SIAM J. Comput., 47(2):493–523, 2018. doi:10.1137/17M1129088.

[12] Prahladh Harsha and Srikanth Srinivasan. On polynomial approximations to AC. Random Struct. Algorithms, 54(2):289–303, 2019. doi:10.1002/rsa.20786.

[13] Johan Håstad. On the correlation of parity and small-depth circuits. SIAM J. Comput., 43(5):1699–1708, 2014. doi:10.1137/120897432.

[14] John Hastad. Almost optimal lower bounds for small depth circuits. Adv. Comput. Res., 5:143–170, 1989.

[15] Pooya Hatami, William Hoza, Avishay Tal, and Roei Tell. Fooling constant-depth threshold circuits. Electron. Colloquium Comput. Complex., page 2, 2021. URL: https://eccc.weizmann. ac.il/report/2021/002.

[16] Russell Impagliazzo, William Matthews, and Ramamohan Paturi. A satisfiability algorithm for ac0 . In SODA, pages 961–972. SIAM, 2012.

[17] Russell Impagliazzo, Raghu Meka, and David Zuckerman. Pseudorandomness from shrinkage. In 53rd Annual IEEE Symposium on Foundations of Computer Science, FOCS 2012, New Brunswick, NJ, USA, October 20-23, 2012, pages 111–119. IEEE Computer Society, 2012. doi:10.1109/FOCS.2012.78.

[18] Zander Kelley. An improved derandomization of the switching lemma. In Samir Khuller and Virginia Vassilevska Williams, editors, STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021, pages 272–282. ACM, 2021. doi:10.1145/3406325.3451054.

[19] Chin Ho Lee and Emanuele Viola. More on bounded independence plus noise: Pseudorandom generators for read-once polynomials. Theory Comput., 16:1–50, 2020. doi:10.4086/toc. 2020.v016a007.

[20] Nathan Linial, Yishay Mansour, and Noam Nisan. Constant depth circuits, fourier transform, and learnability. J. ACM, 40(3):607–620, 1993.

[21] Michael Luby and Boban Velickovic. On deterministic approximation of DNF. Algorithmica, 16(4/5):415–433, 1996. doi:10.1007/BF01940873.

[22] Raghu Meka, Omer Reingold, and Avishay Tal. Pseudorandom generators for width-3 branching programs. In Moses Charikar and Edith Cohen, editors, Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Phoenix, AZ, USA, June 23-26, 2019, pages 626–637. ACM, 2019. doi:10.1145/3313276.3316319.

[23] Raghu Meka and David Zuckerman. Pseudorandom generators for polynomial threshold functions. SIAM J. Comput., 42(3):1275–1301, 2013. doi:10.1137/100811623.

[24] Noam Nisan and Avi Wigderson. Hardness vs randomness. J. Comput. Syst. Sci., 49(2):149–167, 1994. doi:10.1016/S0022-0000(05)80043-1.

[25] Rocco A. Servedio and Li-Yang Tan. Improved pseudorandom generators from pseudorandom multi-switching lemmas. In Dimitris Achlioptas and László A. Végh, editors, Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RAN- DOM 2019, September 20-22, 2019, Massachusetts Institute of Technology, Cambridge, MA, USA, volume 145 of LIPIcs, pages 45:1–45:23. Schloss Dagstuhl - Leibniz-Zentrum für Inform- atik, 2019. doi:10.4230/LIPIcs.APPROX-RANDOM.2019.45.

[26] Avishay Tal. Tight bounds on the fourier spectrum of AC0. In Ryan O'Donnell, editor, 32nd Computational Complexity Conference, CCC 2017, July 6-9, 2017, Riga, Latvia, volume 79 of LIPIcs, pages 15:1–15:31. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. doi:10.4230/LIPIcs.CCC.2017.15.

[27] Luca Trevisan and Tongke Xue. A derandomized switching lemma and an improved derandom- ization of AC0. In Proceedings of the 28th Conference on Computational Complexity, CCC 2013, K.lo Alto, California, USA, 5-7 June, 2013, pages 242–247. IEEE Computer Society, 2013. doi:10.1109/CCC.2013.32.

[28] Salil P. Vadhan. Pseudorandomness. Found. Trends Theor. Comput. Sci., 7(1-3):1–336, 2012. doi:10.1561/0400000010.

[29] Emanuele Viola. AC0 unpredictability. ACM Trans. Comput. Theory, 13(1):5:1–5:8, 2021. doi:10.1145/3442362.